

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I Sameer Ahmed G (192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameer Ahmed G

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Book {

    private int bookId;

    private String title;

    private String author;

    private double price;

    private boolean issued;

    Book(int bookId, String title, String author, double price) {

        this.bookId = bookId;

        this.title = title;

        this.author = author;

        this.price = price;

        issued = false;

    }

    public int getBookId() {

        return bookId;

    }

    public void issueBook() {

        if (!issued) {

            issued = true;
```

```

        System.out.println("Book Issued Successfully.");
    } else {
        System.out.println("Book Already Issued.");
    }
}

public void returnBook() {
    if (issued) {
        issued = false;
        System.out.println("Book Returned Successfully.");
    } else {
        System.out.println("Book is Already Available.");
    }
}

public void display() {
    System.out.println("Book ID : " + bookId);
    System.out.println("Title : " + title);
    System.out.println("Author : " + author);
    System.out.println("Price : " + price);
    System.out.println("Status : " + (issued ? "Issued" : "Available"));
    System.out.println();
}
}

public class LibraryBookManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```

```
Book[] books = new Book[10];

int count = 0, choice;

do {

    System.out.println("\n1.Add Book");

    System.out.println("2.Issue Book");

    System.out.println("3.Return Book");

    System.out.println("4.Search Book");

    System.out.println("5.Display All");

    System.out.println("6.Exit");

    System.out.print("Enter Choice: ");

    choice = sc.nextInt();

    switch (choice) {

        case 1:

            System.out.print("Book ID: ");

            int id = sc.nextInt();

            sc.nextLine();

            System.out.print("Title: ");

            String title = sc.nextLine();

            System.out.print("Author: ");

            String author = sc.nextLine();

            System.out.print("Price: ");

            double price = sc.nextDouble();

            books[count++] = new Book(id, title, author, price);

            System.out.println("Book Added Successfully.");

            break;

        case 2:
```

```
System.out.print("Enter Book ID: ");
```

```
id = sc.nextInt();
```

```
for (int i = 0; i < count; i++) {
```

```
    if (books[i].getBookId() == id) {
```

```
        books[i].issueBook();
```

```
    }
```

```
}
```

```
break;
```

case 3:

```
System.out.print("Enter Book ID: ");
```

```
id = sc.nextInt();
```

```
for (int i = 0; i < count; i++) {
```

```
    if (books[i].getBookId() == id) {
```

```
        books[i].returnBook();
```

```
    }
```

```
}
```

```
break;
```

case 4:

```
System.out.print("Enter Book ID: ");
```

```
id = sc.nextInt();
```

```
for (int i = 0; i < count; i++) {
```

```
    if (books[i].getBookId() == id) {
```

```
        books[i].display();
```

```
    }
```

```
}
```

```
break;
```

case 5:

```
for (int i = 0; i < count; i++) {
```

```
    books[i].display();
```

```
}
```

```
break;
```

case 6:

```
System.out.println("Program Ended.");
```

```
break;
```

default:

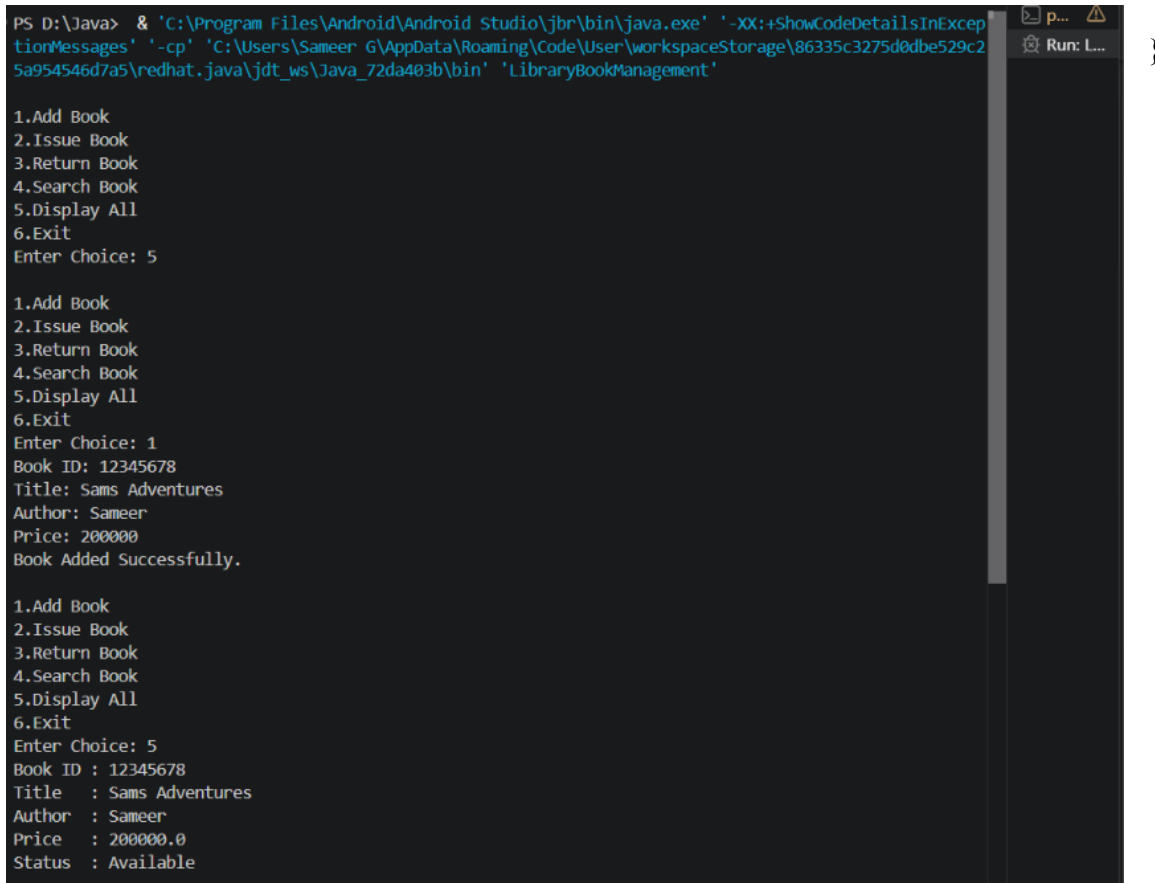
```
System.out.println("Invalid Choice.");
```

```
}
```

```
} while (choice != 6);
```

```
sc.close();
```

```
}
```



```
PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'LibraryBookManagement'
```

```
1.Add Book
2.Issue Book
3.Return Book
4.Search Book
5.Display All
6.Exit
Enter Choice: 5

1.Add Book
2.Issue Book
3.Return Book
4.Search Book
5.Display All
6.Exit
Enter Choice: 1
Book ID: 12345678
Title: Sams Adventures
Author: Sameer
Price: 200000
Book Added Successfully.

1.Add Book
2.Issue Book
3.Return Book
4.Search Book
5.Display All
6.Exit
Enter Choice: 5
Book ID : 12345678
Title : Sams Adventures
Author : Sameer
Price : 200000.0
Status : Available
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Employee {

    int id;

    String name;

    Employee(int id, String name) {

        this.id = id;

        this.name = name;

    }

    void calculateSalary() {

        System.out.println("Salary Calculation");

    }

    void display() {

        System.out.println("Employee ID : " + id);

        System.out.println("Employee Name : " + name);

    }

}

class PermanentEmployee extends Employee {

    double basicSalary;

    PermanentEmployee(int id, String name, double basicSalary) {

        super(id, name);

        this.basicSalary = basicSalary;

    }

}
```

```

    }

    @Override
    void calculateSalary() {
        double salary = basicSalary + (0.20 * basicSalary);

        display();

        System.out.println("Permanent Employee Salary : " + salary);
    }
}

class ContractEmployee extends Employee {
    int hoursWorked;

    double ratePerHour;

    ContractEmployee(int id, String name, int hoursWorked, double ratePerHour) {
        super(id, name);

        this.hoursWorked = hoursWorked;

        this.ratePerHour = ratePerHour;
    }

    @Override
    void calculateSalary() {
        double salary = hoursWorked * ratePerHour;

        display();

        System.out.println("Contract Employee Salary : " + salary);
    }
}

public class EmployeePayrollSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```



```

        System.out.println("1. Permanent Employee");

        System.out.println("2. Contract Employee");

        System.out.print("Enter Choice: ");

        int choice = sc.nextInt();

        System.out.print("Enter Employee ID: ");

        int id = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Employee Name: ");

        String name = sc.nextLine();

        Employee emp;

        if (choice == 1) {

            System.out.print("Enter Basic Salary: ");

            double salary = sc.nextDouble();

            emp = new PermanentEmployee(id, name, salary);

        } else {

            System.out.print("Enter Hours Worked: ");

            int hours = sc.nextInt();

            System.out.print("Enter Rate Per Hour: ");

            double rate = sc.nextDouble();

            emp = new ContractEmployee(id, name, hours, rate);

        }

        emp.calculateSalary();

        sc.close();

    }

}

```

```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer d
\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'EmployeePayrollSystem'
1. Permanent Employee
2. Contract Employee
Enter Choice: 2
Enter Employee ID: 23456
Enter Employee Name: Sameer
Enter Hours Worked: 5
Enter Rate Per Hour: 100
Employee ID : 23456
Employee Name : Sameer
Contract Employee Salary : 500.0
PS D:\Java>

```

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

```

import java.util.Scanner;

class Vehicle {

    String vehicleName;

    Vehicle(String vehicleName) {

        this.vehicleName = vehicleName;

    }

    void calculateRent(int days) {

        System.out.println("Rent Calculation");

    }

}

class Car extends Vehicle {

    Car() {

        super("Car");

    }

}

```

```

@Override

void calculateRent(int days) {

    System.out.println("Vehicle : " + vehicleName);

    System.out.println("Total Rent : " + (days * 1000));

}

}

class Bike extends Vehicle {

    Bike() {

        super("Bike");

    }

    @Override

    void calculateRent(int days) {

        System.out.println("Vehicle : " + vehicleName);

        System.out.println("Total Rent : " + (days * 500));

    }

}

class Bus extends Vehicle {

    Bus() {

        super("Bus");

    }

    @Override

    void calculateRent(int days) {

        System.out.println("Vehicle : " + vehicleName);

        System.out.println("Total Rent : " + (days * 3000));

    }

}

```

```
public class VehicleRentalSystem {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("1. Car");  
  
        System.out.println("2. Bike");  
  
        System.out.println("3. Bus");  
  
        System.out.print("Enter Choice: ");  
  
        int choice = sc.nextInt();  
  
        System.out.print("Enter Number of Days: ");  
  
        int days = sc.nextInt();  
  
        Vehicle v;  
  
        switch (choice) {  
  
            case 1:  
  
                v = new Car();  
  
                break;  
  
            case 2:  
  
                v = new Bike();  
  
                break;  
  
            case 3:  
  
                v = new Bus();  
  
                break;  
  
            default:  
  
                System.out.println("Invalid Choice");  
  
                sc.close();  
  
                return;  
  
        }  
    }  
}
```

```

        v.calculateRent(days);

        sc.close();
    }
}

```

```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'VehicleRentalSystem'
1. Car
2. Bike
3. Bus
Enter Choice: 1
Enter Number of Days: 12
Vehicle : Car
Total Rent : 12000
PS D:\Java>

```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

```

import java.util.Scanner;

class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }
}

```

```

public void deposit(double amount) {

    if (amount > 0) {

        balance += amount;

        System.out.println("Amount Deposited Successfully.");

    } else {

        System.out.println("Invalid Amount.");

    }

}

public void withdraw(double amount) {

    if (amount > 0 && amount <= balance) {

        balance -= amount;

        System.out.println("Amount Withdrawn Successfully.");

    } else {

        System.out.println("Insufficient Balance.");

    }

}

public void displayBalance() {

    System.out.println("Account Number : " + accountNumber);

    System.out.println("Account Holder : " + holderName);

    System.out.println("Current Balance : " + balance);

}

}

public class BankAccountSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Account Number: ");
    }
}

```

```
int accNo = sc.nextInt();

sc.nextLine();

System.out.print("Enter Account Holder Name: ");

String name = sc.nextLine();

System.out.print("Enter Initial Balance: ");

double balance = sc.nextDouble();

BankAccount account = new BankAccount(accNo, name, balance);

System.out.println("\n1. Deposit");

System.out.println("2. Withdraw");

System.out.print("Enter Choice: ");

int choice = sc.nextInt();

System.out.print("Enter Amount: ");

double amount = sc.nextDouble();

if (choice == 1) {

    account.deposit(amount);

} else if (choice == 2) {

    account.withdraw(amount);

} else {

    System.out.println("Invalid Choice");

}

account.displayBalance();

sc.close();

}

}
```

```
PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G
\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'BankAccountSystem'
Enter Account Number: 12345678
Enter Account Holder Name: Sameer
Enter Initial Balance: 10000

1. Deposit
2. Withdraw
Enter Choice: 1
Enter Amount: 2000
Amount Deposited Successfully.
Account Number : 12345678
Account Holder : Sameer
Current Balance : 12000.0
PS D:\Java>
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Admission {

    void calculateFee(double fee) {

        System.out.println("Undergraduate Fee : " + fee);

    }

    void calculateFee(double fee, int years) {

        System.out.println("Postgraduate Fee : " + (fee * years));

    }

    void calculateFee(double fee, double scholarship) {

        double finalFee = fee - scholarship;

        System.out.println("Scholarship Student Fee : " + finalFee);

    }

}
```



```
public class UniversityAdmissionSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Admission a = new Admission();

        System.out.println("1. Undergraduate");

        System.out.println("2. Postgraduate");

        System.out.println("3. Scholarship Student");

        System.out.print("Enter Choice: ");

        int choice = sc.nextInt();

        switch (choice) {

            case 1:

                System.out.print("Enter Undergraduate Fee: ");

                double ugFee = sc.nextDouble();

                a.calculateFee(ugFee);

                break;

            case 2:

                System.out.print("Enter Annual Fee: ");

                double pgFee = sc.nextDouble();

                System.out.print("Enter Number of Years: ");

                int years = sc.nextInt();

                a.calculateFee(pgFee, years);

                break;

            case 3:

                System.out.print("Enter Total Fee: ");

                double fee = sc.nextDouble();

                System.out.print("Enter Scholarship Amount: ");
```

```

        double scholarship = sc.nextDouble();

        a.calculateFee(fee, scholarship);

        break;

    default:

        System.out.println("Invalid Choice");

    }

    sc.close();

}
}

```

```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'UniversityAdmissionSystem'
1. Undergraduate
2. Postgraduate
3. Scholarship Student
Enter Choice: 2
Enter Annual Fee: 1000000
Enter Number of Years: 5
Postgraduate Fee : 5000000.0
PS D:\Java>

```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

```

import java.util.Scanner;

abstract class Shape {

    abstract void area();

}

class Circle extends Shape {

    double radius;

```

```

Circle(double radius) {

    this.radius = radius;

}

@Override

void area() {

    System.out.println("Area of Circle : " + (3.14 * radius * radius));

}

}

class Rectangle extends Shape {

    double length, breadth;

    Rectangle(double length, double breadth) {

        this.length = length;

        this.breadth = breadth;

    }

    @Override

    void area() {

        System.out.println("Area of Rectangle : " + (length * breadth));

    }

}

class Triangle extends Shape {

    double base, height;

    Triangle(double base, double height) {

        this.base = base;

        this.height = height;

    }

    @Override

```

```

void area() {

    System.out.println("Area of Triangle : " + (0.5 * base * height));

}

}

public class ShapeAreaCalculator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Circle");

        System.out.println("2. Rectangle");

        System.out.println("3. Triangle");

        System.out.print("Enter Choice: ");

        int choice = sc.nextInt();

        Shape s;

        switch (choice) {

            case 1:

                System.out.print("Enter Radius: ");

                double r = sc.nextDouble();

                s = new Circle(r);

                break;

            case 2:

                System.out.print("Enter Length: ");

                double l = sc.nextDouble();

                System.out.print("Enter Breadth: ");

                double b = sc.nextDouble();

                s = new Rectangle(l, b);

                break;

```

case 3:

```
System.out.print("Enter Base: ");

double base = sc.nextDouble();

System.out.print("Enter Height: ");

double h = sc.nextDouble();

s = new Triangle(base, h);

break;
```

default:

```
System.out.println("Invalid Choice");

sc.close();

return;
```

```
}
```

```
s.area();
```

```
sc.close();
```

```
}
```

```
}
```

```
PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G
\Roaming\Code\User\workspaceStorage\86335c3275d0d0be529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'BankAccountSystem'
Enter Account Number: 12345678
Enter Account Holder Name: Sameer
Enter Initial Balance: 10000

1. Deposit
2. Withdraw
Enter Choice: 1
Enter Amount: 2000
Amount Deposited Successfully.
Account Number : 12345678
Account Holder : Sameer
Current Balance : 12000.0
PS D:\Java>
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
interface MedicalRecord {  
  
    void addRecord();  
  
    void displayRecord();  
  
}
```

```
class Patient implements MedicalRecord {  
  
    int id;  
  
    String name;  
  
    String disease;  
  
    public void addRecord() {  
  
        Scanner sc = new Scanner(System.in);  
  
  
        System.out.print("Enter Patient ID: ");  
  
        id = sc.nextInt();  
  
        sc.nextLine();  
  
  
        System.out.print("Enter Patient Name: ");  
  
        name = sc.nextLine();  
  

```

```
        System.out.print("Enter Disease: ");

        disease = sc.nextLine();

    }
```

```
    public void displayRecord() {

        System.out.println("\nPatient Record");

        System.out.println("Patient ID : " + id);

        System.out.println("Patient Name : " + name);

        System.out.println("Disease : " + disease);

    }

}
```

```
class Doctor implements MedicalRecord {

    int id;

    String name;

    String specialization;

    public void addRecord() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Doctor ID: ");

        id = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Doctor Name: ");
```

```
name = sc.nextLine();
```

```
System.out.print("Enter Specialization: ");
```

```
specialization = sc.nextLine();
```

```
}
```

```
public void displayRecord() {
```

```
    System.out.println("\nDoctor Record");
```

```
    System.out.println("Doctor ID : " + id);
```

```
    System.out.println("Doctor Name : " + name);
```

```
    System.out.println("Specialization : " + specialization);
```

```
}
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("1. Patient");
```

```
        System.out.println("2. Doctor");
```

```
        System.out.print("Enter Choice: ");
```

```
        int choice = sc.nextInt();
```

```
        MedicalRecord record;
```



```

        if (choice == 1) {

            record = new Patient();

        } else {

            record = new Doctor();

        }

        record.addRecord();

        record.displayRecord();

        sc.close();

    }

}

```

```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData\Roaming\Cod
e\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'Main'
Enter Product ID: 12345678
Enter Product Name: Bottle
Enter Product Price: 123
Enter Quantity: 4

----- BILL -----
Product ID : 12345678
Product Name : Bottle
Price : 123.0
Quantity : 4
Total Amount : 492.0
PS D:\Java>

```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Product {
```

```
    int productId;
```

```
    String productName;
```

```
    double price;
```

```
    Product(int productId, String productName, double price) {
```

```
        this.productId = productId;
```

```
        this.productName = productName;
```

```
        this.price = price;
```

```
    }
```

```
    void displayProduct() {
```

```
        System.out.println("Product ID : " + productId);
```

```
        System.out.println("Product Name : " + productName);
```

```
        System.out.println("Price : " + price);
```

```
    }
```

```
}
```

```
class Order {  
  
    Product product;  
  
    int quantity;  
  
    Order(Product product, int quantity) {  
  
        this.product = product;  
  
        this.quantity = quantity;  
  
    }  
  
    void generateBill() {  
  
        product.displayProduct();  
  
        System.out.println("Quantity : " + quantity);  
  
        System.out.println("Total Amount : " + (product.price * quantity));  
  
    }  
}
```

```
class Main {  
  
    public static void main(String[] args) {  
  
  
  
        Scanner sc = new Scanner(System.in);  
  
  
  
        System.out.print("Enter Product ID: ");  
  
        int id = sc.nextInt();  
  
        sc.nextLine();  
  

```

```

        System.out.print("Enter Product Name: ");

        String name = sc.nextLine();

        System.out.print("Enter Product Price: ");

        double price = sc.nextDouble();

        System.out.print("Enter Quantity: ");

        int quantity = sc.nextInt();

        Product p = new Product(id, name, price);

        Order o = new Order(p, quantity);

        System.out.println("\n----- BILL -----");

        o.generateBill();

        sc.close();

    }

}

```

```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData\Roaming\Cod
e\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'Main'
Enter Product ID: 12345678
Enter Product Name: Bottle
Enter Product Price: 123
Enter Quantity: 4

----- BILL -----
Product ID : 12345678
Product Name : Bottle
Price : 123.0
Quantity : 4
Total Amount : 492.0
PS D:\Java>

```

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

```
class Student {
```

```
    int rollNo;
```

```
    String name;
```

```
    int m1, m2, m3;
```

```
    int total;
```

```
    double average;
```

```
    String grade;
```

```
    void getData(Scanner sc) {
```

```
        System.out.print("Enter Roll Number: ");
```

```
        rollNo = sc.nextInt();
```

```
        sc.nextLine();
```

```
        System.out.print("Enter Name: ");
```

```
        name = sc.nextLine();
```

```
        System.out.print("Enter Marks in 3 Subjects: ");
```

```
        m1 = sc.nextInt();
```

```
        m2 = sc.nextInt();
```

```
    m3 = sc.nextInt();  
}
```

```
void calculate() {  
    total = m1 + m2 + m3;  
    average = total / 3.0;
```

```
    if (average >= 90)  
        grade = "A";  
    else if (average >= 75)  
        grade = "B";  
    else if (average >= 50)  
        grade = "C";  
    else  
        grade = "F";  
}
```

```
void display() {  
    System.out.println("\nRoll No : " + rollNo);  
    System.out.println("Name : " + name);  
    System.out.println("Total : " + total);  
    System.out.println("Average : " + average);  
    System.out.println("Grade : " + grade);  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter Number of Students: ");  
        int n = sc.nextInt();  
  
        Student[] s = new Student[n];  
  
        for (int i = 0; i < n; i++) {  
            s[i] = new Student();  
            s[i].getData(sc);  
            s[i].calculate();  
        }  
  
        int topper = 0;  
  
        for (int i = 1; i < n; i++) {  
            if (s[i].total > s[topper].total)  
                topper = i;  
        }  
  
        System.out.println("\n----- Student Details -----");  
    }  
}
```

```

        for (int i = 0; i < n; i++) {

            s[i].display();

        }

        System.out.println("\nClass Topper : " + s[topper].name);

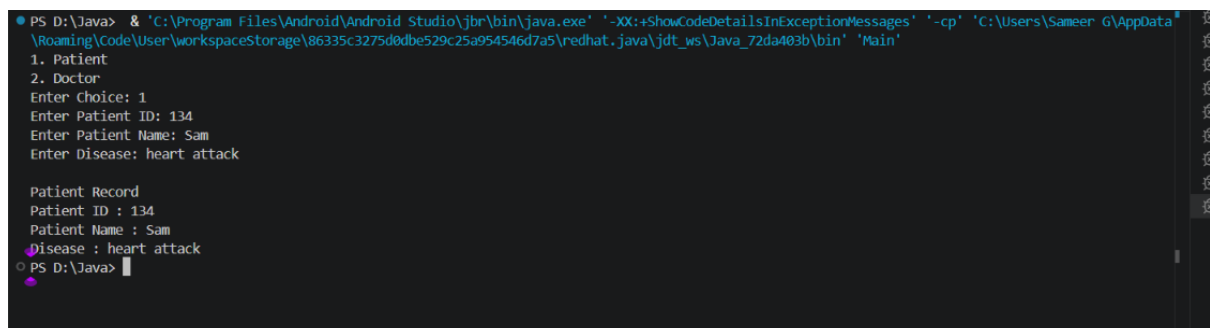
        System.out.println("Top Score : " + s[topper].total);

        sc.close();

    }

}

```



```

PS D:\Java> & 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Sameer G\AppData
\Roaming\Code\User\workspaceStorage\86335c3275d0dbe529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin' 'Main'
1. Patient
2. Doctor
Enter Choice: 1
Enter Patient ID: 134
Enter Patient Name: Sam
Enter Disease: heart attack

Patient Record
Patient ID : 134
Patient Name : Sam
Disease : heart attack
PS D:\Java>

```


10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

```
import java.util.Scanner;
```

```
class Device {  
    String name;  
  
    Device(String name) {  
        this.name = name;  
    }  
  
    void turnOn() {  
        System.out.println(name + " is ON");  
    }  
  
    void turnOff() {  
        System.out.println(name + " is OFF");  
    }  
  
    void powerConsumption() {  
        System.out.println("Power Consumption");  
    }  
}
```

```
class Light extends Device {
```

```
    Light() {  
        super("Light");
```

```
}
```

```
@Override
```

```
void powerConsumption() {
```

```
    System.out.println("Power Consumption : 20 Watts");
```

```
}
```

```
}
```

```
class Fan extends Device {
```

```
    Fan() {
```

```
        super("Fan");
```

```
    }
```

```
@Override
```

```
void powerConsumption() {
```

```
    System.out.println("Power Consumption : 75 Watts");
```

```
}
```

```
}
```

```
class AirConditioner extends Device {
```

```
    AirConditioner() {
```

```
        super("Air Conditioner");
```

```
    }
```

```
@Override
```

```
void powerConsumption() {
```

```
    System.out.println("Power Consumption : 1500 Watts");
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        Device device;
```

```
        System.out.println("1. Light");
```

```
        System.out.println("2. Fan");
```

```
        System.out.println("3. Air Conditioner");
```

```
        System.out.print("Enter Choice: ");
```

```
        int choice = sc.nextInt();
```

```
        switch (choice) {
```

```
            case 1:
```

```
                device = new Light();
```

```
                break;
```

```
            case 2:
```

```
                device = new Fan();
```

```
                break;
```

```
            case 3:
```

```
                device = new AirConditioner();
```

```
                break;
```

```
            default:
```

```
                System.out.println("Invalid Choice");
```

```
                sc.close();
```

```
                return;
```

```
        }
```

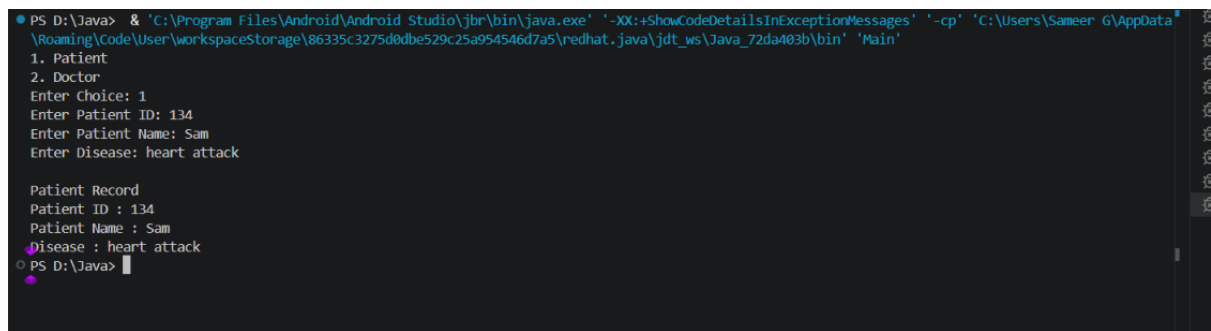
```
        device.turnOn();

        device.powerConsumption();

        device.turnOff();

    }

    sc.close();
}
```



```
PS D:\Java> & "C:\Program Files\Android\Android Studio\jbr\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\Sameer G\AppData
\Roaming\Code\User\workspaceStorage\86335c3275d0db529c25a954546d7a5\redhat.java\jdt_ws\Java_72da403b\bin" "Main"
1. Patient
2. Doctor
Enter Choice: 1
Enter Patient ID: 134
Enter Patient Name: Sam
Enter Disease: heart attack

Patient Record
Patient ID : 134
Patient Name : Sam
Disease : heart attack
PS D:\Java>
```